Reinforcement Learning for Within-Visual-Range Air Combat with Curriculum Learning and Dynamic Opponent Pooling

Yue Yu[1], Yuhang Wang[1], Zhe Zhang[2], Bei Lu[1], Qifu Li[1*]

[1*]School of Aeronautics and Astronautics, Shanghai Jiao Tong University, Shanghai, China.
[2]Chinese Flight Test Establishment, Xi'an, China.

*Corresponding author(s). E-mail(s): qifuli@sjtu.edu.cn;

Abstract

Achieving stable training and robust generalization in autonomous air combat remains a significant challenge for unmanned combat aircraft. This study proposes a reinforcement learning framework for within-visual-range dogfights in a six-degree-of-freedom air combat simulation environment based on JSBSim. A proximal policy optimization-based architecture is developed, featuring curriculum learning and a dynamic opponent pool. The curriculum progressively shifts the learning objective from stable flight control to tactical air-combat behavior, while gradually increasing opponent threat levels to alleviate sparse-reward exploration difficulty. The dynamic opponent pool combines protected opponents with competitive historical policy snapshots, improving adversarial diversity and mitigating policy overfitting during self-play. Engagement simulations demonstrate that the proposed framework achieves stable convergence and improved tactical performance, with the best proposed model improving the win rate over the baseline by 55.2 percentage points.

Keywords: reinforcement learning, within-visual-range air combat, dogfight, curriculum learning, dynamic opponent pool

1 Introduction

Recent advances in artificial intelligence have driven air combat away from rule-based systems and toward data-driven approaches [1]. Dogfight scenarios are especially challenging because they require aircraft to interpret high-dimensional states and generate continuous control commands at high frequency under rapidly changing conditions. Reinforcement learning (RL) improves behavior through repeated trial-and-error in the environment, making it well suited to dynamic adversarial settings. The potential of RL for dogfight has been supported by recent studies. A representative example is the 2020 Defense Advanced Research Projects Agency (DARPA) AlphaDogfight Trials (ADT), in which an artificial intelligence (AI) agent defeated an experienced human pilot in a one-on-one within-visual-range (WVR) dogfight [2]. This result suggested that AI could achieve competitive performance in tactical decision-making and maneuvering control, and it further stimulated research on autonomous unmanned combat aircraft.

Even so, robust performance in dogfight scenarios remains difficult to achieve. Specifically, the three major challenges are the training instability inherent in high-dimensional continuous control, the exploration difficulty under sparse rewards, and the policy overfitting caused by self-play.

The high-dimensional and continuous nature of aircraft dynamics often leads to convergence difficulty during training. To alleviate this, many studies have adopted hierarchical architectures to decouple complex control tasks. For instance, at the low-level maneuvering layer, RL-based controllers have outperformed conventional proportional-integral-derivative (PID) controllers in response speed and tracking accuracy, while also showing stronger adaptability to severe wind disturbances and aggressive maneuvers [3–7]. Similarly, to address the curse of dimensionality and the credit assignment problem in multi-aircraft cooperation, hierarchical architectures and value decomposition methods have been used [8–11]. Methods including hierarchical control and gain scheduling have also been proposed to address convergence difficulty and oscillatory behavior in end-to-end control [12]. Notably, one study based on JSBSim, an open-source flight dynamics model widely used for aircraft simulation [13], demonstrated the effectiveness of a hierarchical RL architecture in experiments against a human pilot [2]. While hierarchical methods simplify the learning process, they often require manual design and may introduce sub-optimalities at the interfaces between decision-making and control layers. Consequently, end-to-end control frameworks are desirable for their potential to learn globally optimal behaviors, though achieving stable convergence remains challenging. To improve end-to-end training stability and sample efficiency, researchers have proposed progressive multi-objective strategy optimization methods [14], as well as model-based reinforcement learning [15]. Furthermore, sequence models such as Transformers and long short-term memory networks (LSTMs) have been integrated to enhance long-horizon decision-making and state estimation under partial observability [16–19].

Beyond control stability, dogfight tasks typically suffer from exploration difficulties due to sparse reward signals. To alleviate this issue and improve RL performance, curriculum learning and imitation learning have been actively explored [20, 21]. The performance of imitation learning remains limited. For instance, it has been observed

that even when facing non-adversarial opponents performing basic maneuvers such as level flight, circling, or serpentine patterns, the shooting success rates achieved through imitation learning still fall short of those seen in expert demonstrations [21]. Studies employing curriculum learning progressively expose agents to increasingly challenging scenarios, such as gradually reducing initial tactical advantages or introducing more competitive opponents [19, 22–24]. Therefore, curriculum learning serves as an effective approach to smoothly guiding the exploration process and overcoming the sparse reward challenge in complex dogfight scenarios.

Within a reinforcement learning framework, improving generalization requires addressing policy overfitting and cyclic dominance that often arise during self-play [25, 26]. To promote gradual and stable performance improvement, one study employed simulated annealing to design a dynamic opponent-pool update rule: a new policy was added to the opponent pool only when its win rate exceeded 50%, the oldest policy was removed, and the sampling distribution was progressively shifted toward stronger opponents [26]. Another study approached the problem from the perspective of population diversity, introducing a population-entropy regularization term to enlarge the KL divergence among policies in the opponent pool [25]. While this entropy-based method improved the generalization of the main agent, its validation was limited to a low-fidelity three-degree-of-freedom point-mass model with a discrete action space. Building on these insights, designing a robust dynamic opponent pool that effectively balances diversity and competence in high-fidelity, continuous-action environments remains essential. Furthermore, a challenge in such frameworks is the cold-start problem, which concerns the selection of appropriate initial opponents to kickstart the learning process. Whether utilizing expert strategies [21] or pre-trained models [25] as baseline adversaries, the competence and diversity of these initial opponents can significantly influence the convergence stability and the ultimate proficiency of the agent.

While RL shows great promise for air combat, convergence under sparse rewards, cyclic dominance, and overfitting in self-play continue to be important open problems. These intertwined challenges form the central focus of the present study.

This work presents a reinforcement learning framework for one-on-one within-visual-range gun-based dogfights, and evaluates it in a six-degree-of-freedom (6-DOF) air combat environment built on JSBSim [13]. The environment models an F-16 aircraft, and adopts engagement criteria consistent with the DARPA setting reported in [2]. Proximal policy optimization (PPO) is used as the underlying learning algorithm. To address the sparse rewards and overfitting tendencies commonly encountered in self-play air combat training, the framework incorporates a curriculum learning framework and a dynamic opponent pool.

The main contributions of this work are summarized as follows:

- A curriculum learning framework is developed, consisting of an objective curriculum that transitions from stable flight control to air-combat objectives, and an opponent curriculum in which adversary difficulty increases progressively over training.
- A dynamic opponent pool is introduced to maintain policy diversity through evaluation during training. In addition, a goalpoint-based RL opponent is incorporated to mitigate the cold-start problem.

The remainder of this paper is organized as follows. Section 2 presents the mathematical formulation of the air combat task and the construction of the simulation environment. Section 3 describes the PPO-based training framework, including the curriculum and the dynamic opponent-pool update strategy. Section 4 evaluates the tactical performance of the trained agent across multiple engagement scenarios. Section 5 concludes the paper.

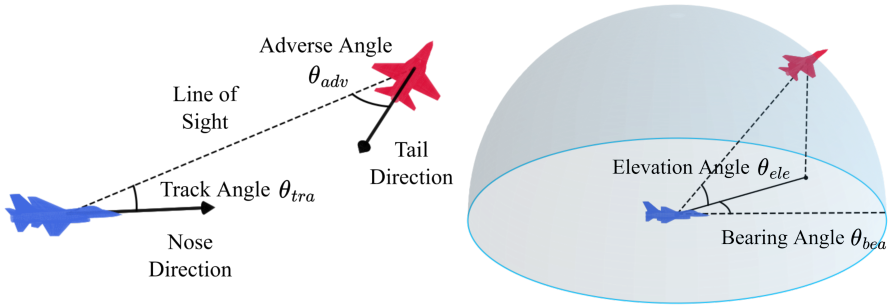
2 Environment Setup

2.1 Air Combat Formulation

WVR air combat is a dynamic and nonlinear adversarial process. In the one-on-one WVR gun dogfight scenario considered in this study, two aircraft execute complex tactical maneuvers in a continuous three-dimensional space in order to secure a favorable firing position behind the opponent. Under a gun-only weapon configuration, the aircraft must avoid incoming fire while closing on the opponent and satisfying range constraints to achieve a valid kill. This setting requires the policy to handle long-horizon spatiotemporal interactions under rapidly changing engagement conditions.

Under a fixed opponent policy, the dogfight process can be approximated from the perspective of the agent as a Markov decision process (MDP), represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$. Here, the state space $\mathcal{S}$ and action space $\mathcal{A}$ define the interaction interface between the aircraft and the environment, while $\mathcal{P}$ denotes the transition function and $\mathcal{R}$ denotes the reward function.

The state space $\mathcal{S}$ provides the agent with a structured description of the current engagement and the aircraft's own flight state. In this work, the observation is represented as a continuous vector. Table 1 summarizes the observation variables, while their full definitions and bounds are provided in Table A1. The angles of relative geometry are shown in Figure 1.



(a) Track angle and adverse angle

(b) Elevation angle and bearing angle

Fig. 1 Definition of angular variables in air combat geometry.

Instead of using a hierarchical control architecture based on high-level waypoints or attitude-command decomposition, this work adopts a mapping from observations

Table 1 Definitions of key state variables in the observation space.

Category	Variable symbol and meaning
Ego Basic States	h, Altitude u, v, w, Body-axis velocities p, q, r, Body-axis angular velocities θ, ϕ, Pitch and roll angles
Relative Geometry	D, Relative distance V_c, Closure rate θ_{tra}, Track angle θ_{adv}, Adverse angle θ_{bea}, Bearing angle θ_{ele}, Elevation angle
Opponent Features	$\mathbf{x}_{oppo}$, Ego basic states and relative geometry from the opponent's perspective
Action Variables	$\delta_a, \delta_e, \delta_r$, Aileron, elevator, and rudder commands δ_T, Throttle command

directly to pilot inputs. Accordingly, the action space $\mathcal{A}$ is defined as a four-dimensional continuous vector, whose components correspond to normalized target roll rate, target pitch rate, target yaw rate, and throttle commands.

The transition function $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ is governed by the equations of motion evaluated by JSBSim. Episode termination is jointly determined by weapon effectiveness, flight-safety constraints, and a maximum engagement duration. Specifically, an episode terminates when an aircraft falls within the opponent's gun engagement zone and its health points (HP) are reduced to zero. In addition, if an aircraft crashes into the ground during aggressive maneuvering, that is, when the altitude satisfies $h \leq 0$, the episode is forcibly terminated. Finally, to prevent prolonged circling at long range or static stalemates, the simulator imposes a maximum episode length of 60 seconds of physical time. Once this limit is reached, the engagement is truncated and scored as a draw.

2.2 Simulation Environment

This work constructs the training environment using JSBSim, an open-source high-fidelity flight dynamics engine. The open-source project Gym-jsbsim [27] provides a viable approach to wrapping JSBSim into a Gymnasium-compatible environment. During simulation, the environment state is updated at a frequency of 60 Hz.

The core dynamics of the system are governed by the real-time integration of a set of ordinary differential equations. The translational equations of motion can be written as

$$\begin{cases} \dot{u} = rv - qw + \frac{1}{m}\left(W_x + F_x^{(A)} + F_x^{(T)}\right) \\ \dot{v} = -ru + pw + \frac{1}{m}\left(W_y + F_y^{(A)} + F_y^{(T)}\right) \\ \dot{w} = qu - pv + \frac{1}{m}\left(W_z + F_z^{(A)} + F_z^{(T)}\right) \end{cases}, \tag{1}$$

where u, v, w and p, q, r denote the translational velocities and angular rates along the three body axes, respectively; m is the aircraft mass; $W_{x,y,z}$ are the gravity components; and $F^{(A)}$ and $F^{(T)}$ represent the resultant aerodynamic forces and engine thrust, respectively. Correspondingly, the rotational equations of motion are given by

$$\begin{cases} \dot{p} = (C_1 r + C_2 p)q + C_3 \mathcal{L} + C_4 \mathcal{N} \\ \dot{q} = C_5 pr - C_6(p^2 - r^2) + C_7 \mathcal{M} \\ \dot{r} = (C_8 p - C_2 r)q + C_4 \mathcal{L} + C_9 \mathcal{N} \end{cases}, \tag{2}$$

where $\mathcal{L}$, $\mathcal{M}$, and $\mathcal{N}$ denote the total rolling, pitching, and yawing moments generated by aerodynamic and propulsive effects. The coefficients C_1 to C_9 are determined entirely by the rigid-body inertia properties of the aircraft, which are defined as follows.

Let $Ox_B y_B z_B$ denote the body-fixed frame, whose origin O is located at the aircraft centre of gravity, with the x_B axis pointing forward along the fuselage reference line, the y_B axis pointing to the right wing, and the z_B axis pointing downward. The inertia tensor of the aircraft expressed in this frame is

$$\mathbf{J} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix}, \tag{3}$$

where (x, y, z) denote the coordinates of a mass element $\mathrm{d}m$ in the body frame, the diagonal entries

$$I_{xx} = \int \left(y^2 + z^2\right) \mathrm{d}m, \qquad I_{yy} = \int \left(x^2 + z^2\right) \mathrm{d}m, \qquad I_{zz} = \int \left(x^2 + y^2\right) \mathrm{d}m \tag{4}$$

are the moments of inertia about the x_B, y_B, and z_B axes, respectively, and the off-diagonal entries

$$I_{xy} = \int xy \, \mathrm{d}m, \qquad I_{xz} = \int xz \, \mathrm{d}m, \qquad I_{yz} = \int yz \, \mathrm{d}m \tag{5}$$

are the corresponding products of inertia. All six quantities are taken about the centre of gravity and have units of slug $\cdot$ ft^2. Since the aircraft is symmetric about its x_B–z_B plane, the products of inertia I_{xy} and I_{yz} vanish, so that I_{xz} is the only non-zero off-diagonal element of $\mathbf{J}$. As $\mathbf{J}$ is symmetric and positive definite, the quantity $\Gamma =$

$I_{xx}I_{zz} - I_{xz}^2$ is strictly positive. With these definitions, the coefficients are expanded as

$$\begin{aligned} C_1 &= \frac{(I_{yy} - I_{zz})I_{zz} - I_{xz}^2}{\Gamma}, \quad C_2 = \frac{(I_{xx} - I_{yy} + I_{zz})I_{xz}}{\Gamma}, \\ C_3 &= \frac{I_{zz}}{\Gamma}, \quad C_4 = \frac{I_{xz}}{\Gamma}, \quad C_5 = \frac{I_{zz} - I_{xx}}{I_{yy}}, \quad C_6 = \frac{I_{xz}}{I_{yy}}, \\ C_7 &= \frac{1}{I_{yy}}, \quad C_8 = \frac{(I_{xx} - I_{yy})I_{xx} + I_{xz}^2}{\Gamma}, \quad C_9 = \frac{I_{xx}}{\Gamma}. \end{aligned} \tag{6}$$

For the F-16 model used in this work, these inertia quantities are updated by JSBSim [13] at every integration step from the airframe inertia tensor together with the pilot and fuel point masses, and therefore depend on the instantaneous loading state. At the loading state used for the initial conditions of the engagements, that is, with an initial fuel load of 3000 lb giving a total weight of 20,630 lb and $m = 641.2$ slugs, the moments and the product of inertia, expressed in slug $\cdot$ ft^2, are

$$I_{xx} = 1.229 \times 10^4, \quad I_{yy} = 5.711 \times 10^4, \quad I_{zz} = 6.707 \times 10^4, \quad I_{xz} = 1.060 \times 10^3. \tag{7}$$

The values given in Eq. (7) vary with the fuel state; when the fuel is exhausted ($m = 548.0$ slugs, that is, 17,630 lb), they become $I_{xx} = 9.50 \times 10^3$, $I_{yy} = 5.682 \times 10^4$, $I_{zz} = 6.411 \times 10^4$, and $I_{xz} = 9.46 \times 10^2$ slug $\cdot$ ft^2. The mass m in Eq. (1) is updated in the same manner.

JSBSim includes an inner-loop fly-by-wire (FBW) architecture. Unlike studies that directly output control-surface deflection commands, the continuous action vector output by the agent, $\mathbf{u}_{cmd} = [u_{roll}, u_{pitch}, u_{yaw}, u_T]^T$, represents pilot-level stick and throttle inputs that serve as target rate and G-load setpoints. Given the current flight state $\mathbf{x}$, the FBW system maps these reference signals into the actual actuator-level surface deflections and engine commands, denoted by $[\delta_a, \delta_e, \delta_r, \delta_T]^T$. This design is consistent with the human–machine interaction logic of a real cockpit and also allows the reinforcement learning agent to benefit implicitly from the built-in stability augmentation loops, thereby focusing on higher-level tactical decision-making rather than high-frequency disturbance rejection at the control-surface level.

The weapon system and engagement criteria used in this work are inspired by the publicly available rules of the DARPA ADT. The environment defines a weapons engagement zone (WEZ) in front of the aircraft nose, shown in Figure 2. Let the position of the ego aircraft be $\mathbf{p}_{agt}$ and that of the opponent aircraft be $\mathbf{p}_{oppo}$. The relative line-of-sight vector is defined as $\mathbf{d} = \mathbf{p}_{oppo} - \mathbf{p}_{agt}$, and the relative Euclidean distance is $D = \|\mathbf{d}\|_2$. If the unit nose-direction vector of the ego aircraft is denoted by $\mathbf{v}_{nose}$, then the track angle θ_{tra} is computed from the inner product as

$$\theta_{tra} = \arccos\left(\frac{\mathbf{d} \cdot \mathbf{v}_{nose}}{\|\mathbf{d}\|_2}\right). \tag{8}$$

The opponent is considered to be inside the WEZ only when it lies within the effective firing range (500 ft $\leq D \leq$ 3000 ft) and inside a cone with an aperture angle of 15°.

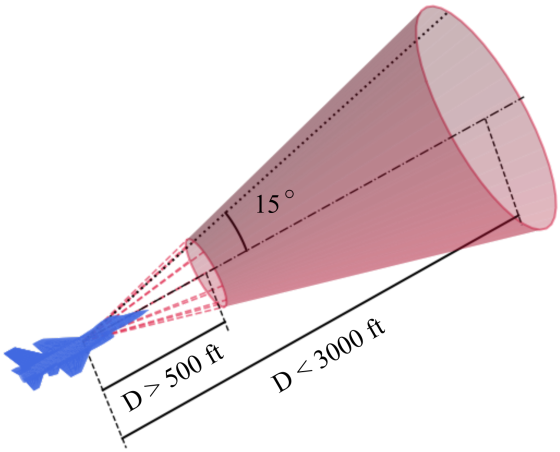


Fig. 2 Definition of the weapons engagement zone.

This condition can be expressed by the indicator function $\mathbb{W}_{WEZ}(t)$:

$$\mathbb{W}_{WEZ}(t) = \begin{cases} 1, & \text{if } 500 \leq D(t) \leq 3000 \text{ and } \theta_{tra}(t) \leq 15^\circ \\ 0, & \text{otherwise} \end{cases}. \tag{9}$$

When $\mathbb{W}_{WEZ}(t) = 1$, the agent applies continuous damage to the opponent at the current time step. The decay of the opponent's HP follows Eq. 10:

$$\mathrm{HP}_{oppo}(t + \Delta t) = \max\left(0, \mathrm{HP}_{oppo}(t) - \lambda \cdot \mathbb{W}_{WEZ}(t) \cdot \Delta t\right), \tag{10}$$

where $\lambda = \frac{3000 - D(t)}{2500}$ is the prescribed per-unit-time gun damage rate.

At the same time, aerodynamic data, FBW computation, and combat logic are highly decoupled within the JSBSim framework, so the proposed simulation architecture has good extensibility. By replacing the aerodynamic database and model configuration, the environment can be extended to other aircraft types, such as the F-15, providing a basis for future studies on multi-aircraft and asymmetric air combat.

2.3 Parallel Sampling Architecture

In RL, environment sampling throughput is often the key bottleneck that limits training efficiency. From the perspective of the agent, the opponent policy should be treated as part of the environment transition. However, in practical implementation, directly coupling the opponent network into each simulation time step can substantially reduce training speed. Specifically, JSBSim flight dynamics simulation relies on serial floating-point computation on the CPU, whereas opponent policy inference is better suited to parallel tensor computation on the GPU. If CPU-based dynamics integration and single-sample neural network forward passes are executed alternately at every time step, the system suffers not only from frequent context switching, but

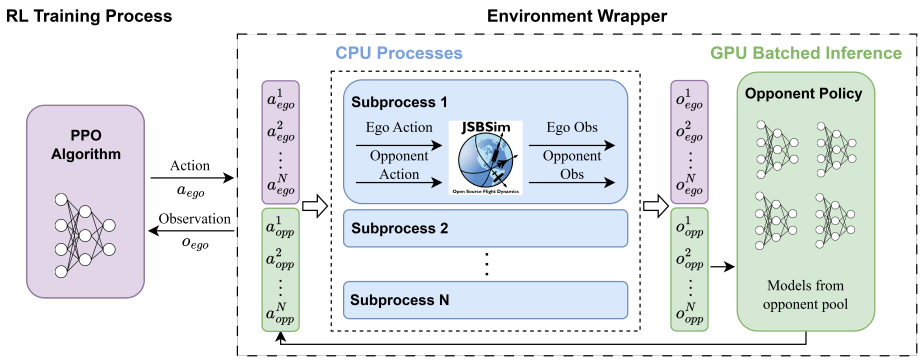


Fig. 3 Multiprocessing parallel acceleration architecture.

also from the inability to exploit the batching advantage of the GPU. As a result, sampling efficiency is significantly degraded by computational serialization.

To address this issue, this work designs a multiprocessing parallel acceleration architecture based on heterogeneous-computing decoupling, whose core idea is to separate dynamics simulation from neural network inference. Figure 3 shows the detail. First, the base environment is refactored into a pure JSBSim dynamics container with no embedded neural network modules. It exposes a state-action interface for both the ego aircraft and the opponent aircraft. Second, by combining CPU multiprocessing with GPU batched inference, a vectorized multi-process environment wrapper is constructed. Each subprocess independently executes flight dynamics simulation. At every interaction step the opponent observations from all sub-environments are collected, concatenated, and sent to the GPU for batched forward inference through the fixed opponent policy network. The resulting opponent actions are then distributed back to the corresponding CPU processes for the next simulation step. Finally, through an external interface abstraction, the wrapper hides the opponent observations and actions from the learning algorithm and exposes only the state and action interfaces of the ego agent. Therefore, under a fixed opponent policy, the overall training procedure remains externally equivalent to a standard single-agent Markov decision process.

This decoupled architecture effectively leverages heterogeneous computing resources, yielding a considerable improvement in simulation throughput on the same hardware platform. On the platform and experiment setup detailed in Section 4, compared with the original serial implementation, whose maximum throughput is only 50 simulation time steps per second, the proposed parallel acceleration structure increases the environment iteration speed to approximately 6700 simulation time steps per second, yielding a 134× speedup. This will provide computational support for subsequent reinforcement learning training.

3 Methods

PPO [28] is a representative Actor–Critic policy optimization method and has been widely applied to decision-making tasks such as robotic control, flight control, and autonomous driving. The core idea of PPO lies in formulating a constrained policy

update objective that prevents the new policy from deviating excessively from the old policy.

Let the current policy parameters be θ and the old policy parameters be θ_{old}. For a state s_t and action a_t, the importance sampling ratio is defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \tag{11}$$

PPO adopts a clipped surrogate objective function:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{12}$$

where $\hat{A}_t$ denotes the advantage function estimate and ϵ is the clipping coefficient. By clipping the importance sampling ratio, this objective restricts the magnitude of the policy change in a single iteration, improving training stability. Meanwhile, a value function network is used to estimate state values and is jointly optimized with the policy network. Combining the policy term, the value term, and an entropy regularization term, the overall optimization objective can be written as

$$L(\theta, \phi) = L^{\text{clip}}(\theta) - c_1 L^{\text{value}}(\phi) + c_2 H(\pi_\theta), \tag{13}$$

where $L^{\text{value}}(\phi)$ represents the value function loss, ϕ is the value network parameters, $H(\pi_\theta)$ is the policy entropy. The weight coefficients c_1 and c_2 are used to balance the contributions of the value-function loss and the entropy regularization, respectively. In practice, c_1 is chosen such that the value loss has a comparable scale to the policy objective and does not dominate the policy update, while c_2 is usually set to a relatively small value to encourage sufficient exploration without preventing policy convergence. These coefficients are typically selected empirically according to the training stability and final performance, and can be tuned through validation experiments.

To estimate the advantage, we employ generalized advantage estimation (GAE), which takes the form

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{14}$$

where V represents the value network, γ is the discount factor and λ is the advantage estimation parameter.

Compared with standard policy gradient methods, it constrains the policy update magnitude, which helps mitigate the instability caused by drastic policy oscillations during training. Therefore, PPO is well suited as the foundational algorithmic framework for air combat control policy learning.

3.1 Baseline

This work adopts PPO as the baseline reinforcement learning algorithm and implements it based on Stable-Baselines3 [29].

We pre-construct a pool of several level-flight opponents. These opponents are all trained using PPO and learn stable flight policies under different reward functions. They are designed to maintain sustained flight under randomly initialized headings without losing control or crashing. These opponents possess only basic flight capability and cannot track targets. Compared with stationary targets or a scripted opponent, this design introduces more realistic flight-dynamics perturbations during baseline training and improves the effectiveness of policy learning.

These level-flight opponents are trained using different combinations of several basic reward terms, which are listed in Table A2.

During baseline training, the agent randomly samples opponents from this fixed opponent pool. In practice, the baseline model can acquire basic target-tracking and kill capabilities against opponents from the training set. Nevertheless, when evaluated against unseen opponents or in self-play tests, the baseline model exhibits weak generalization. In many cases, it fails to locate the target and initiate an attack against opponents outside the training set. In some situations, it even crashes. To address this issue, this work introduces a curriculum learning strategy and a dynamic opponent pool.

3.2 Curriculum Learning

Curriculum learning is commonly used to mitigate the training difficulty caused by sparse rewards. It constructs a task sequence that progresses from simple to complex. In this section, we propose a curriculum learning method composed of an objective curriculum and a dynamic opponent curriculum. The former employs two progressively designed reward functions to ensure that the agent first acquires stable flight control and then gradually develops air combat tactics. The latter adaptively adjusts opponent difficulty to provide progressively increasing game pressure for policy improvement. This design is intended to improve training stability and convergence efficiency.

3.2.1 Objective Curriculum

We implement the objective curriculum by progressively adjusting the reward function. The objective curriculum is structured in two successive stages, where the agent first masters stable flight and then learns air combat skills. The first stage includes only reward terms related to flight quality and does not involve relative engagement geometry or firing-related indicators. The second stage introduces reward terms related to relative geometry and gun engagement, so that the agent can learn tactical positioning and evasive maneuvering. The complete reward definitions for the two stages are given in Table 2 and Table 3, respectively.

The subscript “ub” in Table 2 denotes the upper bound. The penalties of stage one impose direct negative feedback on abrupt control-surface oscillations and large altitude variations, encouraging the agent to maintain near-level flight.

Table 2 Stage-1 reward function.

Reward term	Mathematical expression
Aileron command penalty	$r_{sa} = 4 \cdot \dfrac{0.5}{0.5 + \lvert\delta_a\rvert}$
Throttle command penalty	$r_{st} = 8 \cdot \dfrac{0.1}{0.1 + \lvert\delta_T - 0.4\rvert}$
Roll-angle penalty	$r_{sr} = 4 \cdot \dfrac{0.5}{0.5 + \lvert\phi\rvert}$
Action smoothness penalty	$r_{ap} = 4\left(\Phi_{ap}(s_{t+1}) - \Phi_{ap}(s_t)\right),\ \Phi_{ap}(s) = -\max_j \dfrac{\lvert\Delta u_j\rvert}{u_{j,ub}}$
Altitude stability penalty	$r_{alt} = 8 \times 10^4\left(\Phi_{alt}(s_{t+1}) - \Phi_{alt}(s_t)\right),\ \Phi_{alt}(s) = -\dfrac{\lvert\Delta h\rvert}{h_{ub}}$

Table 3 Stage-2 reward function.

Reward term	Mathematical expression
Relative position reward	$r_{rp} = \left(\dfrac{\theta_{tra}}{\pi} - 2\right)\sigma\left(\dfrac{\theta_{adv}}{\pi}; 18, 0.5\right) - \dfrac{\theta_{tra}}{\pi} + 1$
Closure rate reward	$r_{cr} = \dfrac{V_c}{500}\left[1 - \sigma\left(\dfrac{\theta_{adv}}{\pi}; 18, 0.5\right)\right] \times \sigma\left(D; \dfrac{1}{500}, 2900\right)$
Aiming reward	$r_{gb} = \Gamma_B(D)\left[1 - \sigma\left(\frac{\theta_{tra}}{\pi}; 10^5, \frac{1}{180}\right)\right]$
Opponent aiming penalty	$r_{gr} = -\Gamma_R(D)\,\sigma\left(\frac{\theta_{adv}}{\pi}; 800, \frac{178}{180}\right)$
Low-altitude penalty	$r_{deck} = -4\left[1 - \sigma\left(h; \frac{1}{20}, 1300\right)\right]$

In Table 3, the function $\sigma(x; k, x_0) = 1/(1 + e^{-k(x-x_0)})$ is the logistic function. $\Gamma_B(D)$ and $\Gamma_R(D)$ denote the weighting functions for the gun-effective ranges of the ego and the opponent aircraft, respectively. These functions approach 1 within effective range and decay rapidly to 0 outside that range.

The relative position reward term encourages the agent to occupy a favorable position behind the opponent. The closure rate reward serves to prevent overshoot during the approach. And the two aiming reward terms encourage firing behavior and evasion. Meanwhile, the low-altitude penalty acts as a flight-safety constraint to prevent the agent from entering dangerously low-altitude regimes. This progressive reward design enables the agent to exhibit tactically meaningful offensive and defensive behaviors during combat. The reward function of stage 2 follows the approach adopted in the ADT project [2].

3.2.2 Opponent Curriculum

Objective curriculum cannot regulate the game difficulty during training. If the agent combats with an aggressive opponent from the very beginning, it may be shot down frequently before learning basic evasive maneuvers. As a result, the sparse reward

makes it difficult for the policy network to extract useful gradients from a large number of failed episodes and potentially causing convergence to random behavior. To address this issue, we propose a dynamic-threshold opponent curriculum. By adjusting the opponent's target, the game difficulty increases smoothly.



Fig. 4 Low-threat opponent and high-threat opponent in the opponent curriculum.

Opponent curriculum is achieved by changing how the opponent selects its target. Figure 4 shows a low-threat opponent and a high-threat opponent. Specifically, we define a threshold value $p(t) \in [0, 1]$ that decays linearly with the training step t:

$$p(t) = \max\Big(0,\ 1 - \frac{t}{T}\Big), \tag{15}$$

where T denotes the total number of steps over which the curriculum decays. During the initialization of each training episode, the system samples a random variable ξ uniformly from $(0, 1)$ and assigns a target point to the opponent according to the comparison between ξ and the current threshold $p(t)$:

- If $\xi \leq p(t)$, the opponent is treated as a low-threat opponent. The system generates a target point randomly within the airspace, and the opponent follows a policy that flies toward this target point. In this mode, the opponent does not attack the ego and poses only a minimal threat.
- If $\xi > p(t)$, the opponent is treated as a high-threat opponent. Its target point is updated in real time to the current position of the ego. In this case, the opponent behaves aggressively and continuously attempts to obtain gun-firing opportunities, imposing substantially greater adversarial pressure on the ego.

As training proceeds, $p(t)$ decreases linearly from 1 to 0, so the probability of encountering a high-threat opponent increases correspondingly. At the beginning of training, the agent faces almost exclusively harmless opponents flying toward random points, which allows it to safely explore the flight envelope and accumulate basic flight experience. As the training time increases, high-threat opponents begin to appear with increasing frequency, forcing the policy to gradually learn defensive, evasive, and counter-engagement behaviors. This difficulty scheduling design enables a smooth transition from non-adversarial flight to adversarial dogfighting. It helps alleviate the instability caused by introducing strong opponents too early, while also reducing the risk of policy stagnation induced by prolonged exposure to weak opponents.

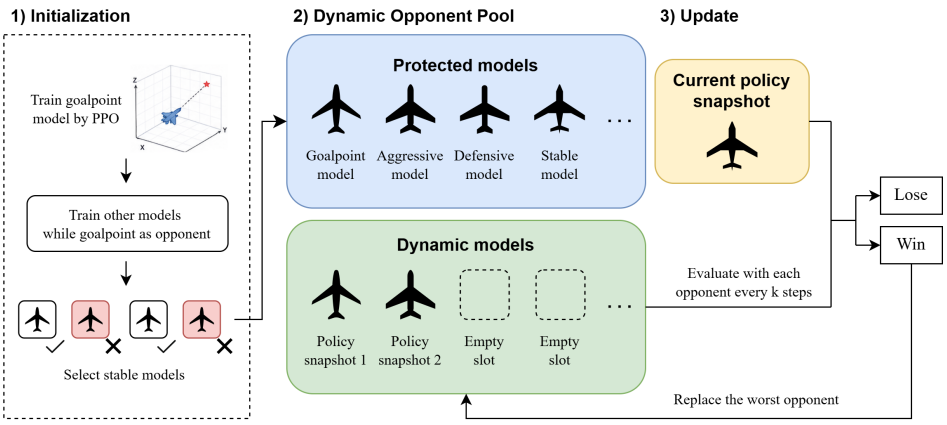


Fig. 5 Dynamic opponent pool.

3.3 Dynamic Opponent Pool

The opponent curriculum described above regulates the training difficulty by changing the opponent's target assignment. However, this mechanism only changes the opponent's behavioral objective during an episode, while the opponent policy itself, i.e., its model parameters and intrinsic combat capability, remains unchanged. Therefore, although opponent curriculum can provide a smooth difficulty transition in the early stage of training, it is insufficient to continuously improve the adversarial competence of the opponent.

To further enhance training, the ego policy should be exposed to opponents with progressively stronger and more diverse combat abilities. Training against a fixed opponent often causes the policy to overfit to a particular behavior pattern, whereas pure self-play may induce policy oscillation due to cyclic dominance. To address this issue, this study proposes a dynamic opponent pool. Fixed benchmark opponents are combined with competitive historical snapshots to provide the policy with a continuously evolving and behaviorally diverse set of adversaries. Details are shown in Figure 5.

3.3.1 Opponent Pool Initialization

The initialization of the opponent pool is critical for stable policy learning. If the initial opponents are overly simple, the agent may overfit to limited behavior patterns, and when stronger opponents are introduced later, the learned policy and value estimates may transfer poorly, leading to performance degradation. Conversely, if the initial opponents are too difficult, the agent may fail to obtain sufficient meaningful interactions. To alleviate this cold-start problem, this paper introduces a goalpoint navigation policy as an initial protected opponent.

The goalpoint model is defined as follows: from an arbitrary initial state, the aircraft will fly toward a randomly generated target point in space. Its observation consists of the ego aircraft state together with the geometric relationship between the aircraft and the target point. Instead of using the absolute coordinates of the

target point, the model takes relative features as input, including relative azimuth, elevation, and distance. This design prevents the policy from memorizing specific spatial locations.

To facilitate the learning of this navigation task, a composite reward function is designed. Four general reward terms are used, namely the aileron command penalty, action smoothness penalty, closure rate reward, and low-altitude penalty, as detailed in Tables 2 and 3. In addition to these terms, the reward function incorporates two task-specific components to actively guide the aircraft: a distance reward and a heading reward. Both components are computed using a scaled asymptotic error form,

$$r_i = \alpha_i \left(1 - \frac{|x_i - x_i^*|/s_i}{|x_i - x_i^*|/s_i + 1}\right) = \alpha_i \frac{s_i}{|x_i - x_i^*| + s_i}, \tag{16}$$

where x_i is the current value of the corresponding variable, x_i^* is the target value, s_i is the scaling factor, and α_i is the component weight.

For the distance reward, the target distance is zero, i.e., $x_i^* = 0$. Let d_t denote the relative distance between the aircraft and the target point in feet. The distance reward is defined as

$$r_{\text{dist}}(t) = 4.0 \cdot \frac{2000}{d(t) + 2000}. \tag{17}$$

This term provides a dense guidance signal that monotonically increases as the relative distance between the aircraft and the target point decreases.

For the heading reward, since the desired heading error is zero, the heading reward is given by

$$r_{\text{head}}(t) = \frac{0.2}{|\theta_{tra}(t)| + 0.2}. \tag{18}$$

This term incentivizes the agent to align its orientation with the target, yielding higher returns when the aircraft flies directly toward the goal point.

In this work, the goalpoint model is pretrained separately using PPO, and the target point is randomly reset at every episode. The resulting goalpoint policy can guide the aircraft toward arbitrary target locations in the airspace using smooth control commands. Using this pretrained goalpoint model as the opponent, a batch of preliminary combat-capable models is trained by applying the reward function detailed in Table 3. These policies are then evaluated according to their training curves, flight safety, and behavioral characteristics. Models with reliable training performance, low crash frequency, clear offensive or evasive behaviors, and diverse maneuvering styles are selected to form the initial protected opponent set. In this way, the initialized opponent pool contains not only the original goalpoint policy, but also several derived opponents that already possess basic maneuvering and combat interaction capability. Compared with initializing the pool using only handcrafted opponents, this procedure provides a broader opponent distribution. Furthermore, because the navigation targets of all opponents within the pool can be flexibly overridden, the system can seamlessly implement the opponent curriculum by toggling opponent behaviors between low threat and high threat. Combined with the objective curriculum, this integrated pipeline systematically guides the agent from basic flight control to complex air combat mastery.

3.3.2 Opponent Pool Structure and Update

Specifically, the proposed framework maintains an opponent pool with fixed capacity. At the beginning of each training episode, an opponent is independently sampled from the pool for each parallel environment. The opponent pool consists of two components. The first is a set of protected opponents initialized using the procedure described in the previous subsection, which provide stable baselines. The second is a set of dynamically updated policy slots used to store historically strong versions of the policy during training. The protected opponents cover representative behavior patterns with different levels of complexity and tactical styles. They remain unchanged throughout training so as to prevent the performance of the opponent pool from degrading due to frequent replacement. In contrast, the dynamic slots are filled progressively as training proceeds and are updated through periodic evaluation.

Let the evaluation interval be denoted by K. Whenever the number of training steps reaches an integer multiple of K, the current policy is frozen to form an evaluation snapshot, which is then matched against every opponent in the pool for N independent episodes. To avoid interference from reward shaping, the update criterion does not rely directly on training return. Instead, it is constructed only from game outcomes and the remaining health at the end of engagement. For opponent i, the corresponding score is defined as

$$S_i = w_i - l_i + 0 \cdot d_i + 0.5 f_i + 0.01\big(hp_i - hp_i^{oppo}\big), \tag{19}$$

where w_i, l_i, and d_i denote the win rate, loss rate, and draw rate, respectively; f_i denotes the opponent self-crash rate; and hp_i and hp_i^{oppo} denote the average remaining health percentages of the ego and the opponent aircraft, respectively. In this way, it offers a more direct measure of actual dogfight effectiveness and reduces the influence of reward hacking.

The update rule is as follows. If the opponent pool still contains empty dynamic slots, the current policy snapshot is inserted directly into one of them. If all dynamic slots are already occupied, the current policy is compared with the replaceable opponents in the pool, and it replaces the weakest one if its performance is superior. The protected opponents are never removed.

By preserving fixed benchmark opponents together with a limited number of high-quality historical snapshots, the dynamic opponent pool helps balance adversarial diversity, training stability, and computational cost. Moreover, the design is compatible with the parallel sampling framework: different sub-environments independently sample opponents at each episode, so that a single batch of experience contains multiple adversarial styles. This design alleviates the bias caused by a single-opponent distribution and improves the generalization of the policy.

In summary, the baseline is trained solely against non-reactive, level-flight opponents that cannot perceive the ego aircraft, and it operates without curriculum learning or a dynamic opponent pool. To achieve better combat performance, our proposed method first constructs an initial opponent pool using the goalpoint model to alleviate the cold-start problem. Subsequently, the policy is trained by integrating

the curriculum learning alongside the dynamic opponent pool to ensure stable learning and robust generalization. The performance of this method will be systematically compared against the baseline in Section 4.

4 Experiments and Evaluation

4.1 Experimental Setup

This section describes the training framework and hardware platform in the experiments. All experiments are implemented based on Stable-Baselines3 and are trained using PPO. Except for the curriculum learning and the opponent selection, all other training configurations are kept identical across different methods to ensure a fair comparison.

The experimental comparison is organized from two perspectives. First, to evaluate the contribution of each proposed component, we start from a baseline and progressively consider the use of curriculum learning and the dynamic opponent pool. This yields four MLP-based models: the baseline model without curriculum learning or dynamic opponent pool (Baseline), the model with the dynamic opponent pool only (BD), the model with curriculum learning only (BC), and the model using both components (BDC-MLP). Second, to study the influence of the policy architecture, we further compare two implementations of the complete BDC method, namely an MLP-based version (BDC-MLP) and an LSTM-based [30] version (BDC-LSTM).

For the component ablation experiments, the MLP-based policy architecture is used by default to ensure that the compared models differ only in the use of curriculum learning and the dynamic opponent pool. For the MLP-based policy, the policy and value networks are parameterized by multilayer perceptrons and share a common feature extractor. For the LSTM-based policy, recurrent layers are further introduced to capture temporal dependencies in the observation sequence. The detailed hyperparameter settings for both architectures are summarized in Tables A3 and A4.

Experiments were conducted on a workstation running Ubuntu 24.04.2 LTS, equipped with an Intel® Core™ i9-14900KF CPU and a NVIDIA GeForce RTX™ 5070 Ti GPU.

4.2 Curriculum Learning Ablation

To evaluate the effect of the curriculum learning on training stability and convergence performance, an ablation study is conducted. The experiments record the evolution of the mean evaluate return of three methods over 3.0×10^8 environment interaction steps. The corresponding training curves are shown in Figure 6.

From the overall trend, the BDC method outperforms the baseline. Although the baseline exhibits some improvement at the early stage of training, it begins to plateau after approximately 5×10^7 steps, with the mean return remaining around 500 for a long period. BDC method converges to approximately the 1000–1250 range, suggesting that curriculum learning and dynamic opponent pool helps improve policy performance.

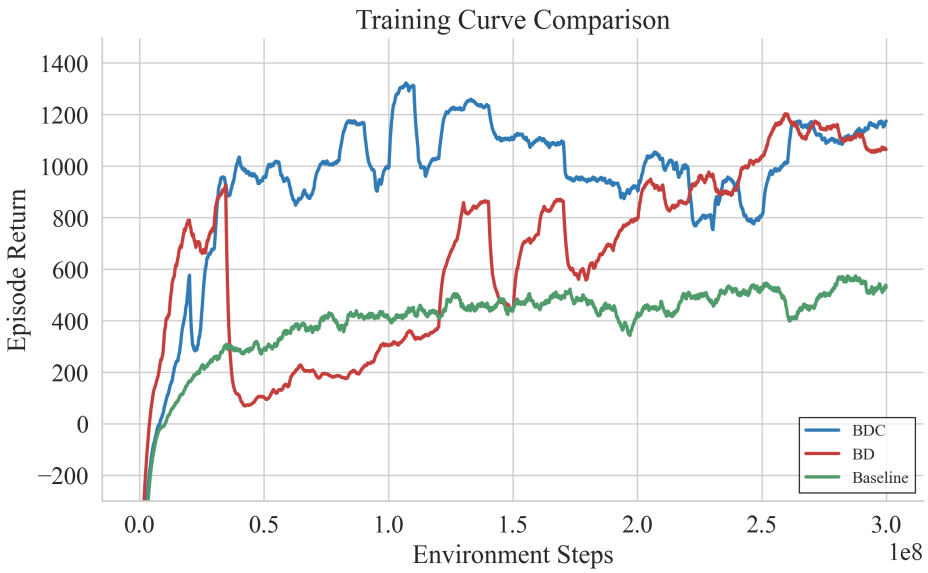


Fig. 6 Episode return of the Baseline, BD, and BDC(MLP) methods. "Baseline" denotes standard PPO with a fixed opponent pool; "D" and "C" represent the Dynamic opponent pool and Curriculum learning components, respectively.

A more direct view of the role of curriculum learning in training stability can be obtained by comparing the BDC method with BD. After removing curriculum learning, the model still benefits from the dynamic opponent pool, but the training curve shows pronounced fluctuations and experiences clear performance collapses around 4×10^7 and 1.3×10^8 steps. During the early stages of training, the agent faces three challenges: complex flight control, air-combat objectives, and an evolving opponent. Forcing the agent to address these simultaneously renders the optimization problem difficult. Ultimately, this may lead to unstable policy updates or even catastrophic forgetting.

By contrast, the BDC method gradually increases task difficulty through the curriculum. This process significantly alleviates the exploration burden at the beginning of training, allowing the model to achieve higher final returns while avoiding large performance oscillations. Although some fluctuations still remain in the later training stage as the dynamic opponent pool continues to evolve, the overall variance is clearly lower than BD method. These results demonstrate that the curriculum learning enhances the stability and robustness of the training process.

4.3 Evaluation

4.3.1 Benchmark Pool Evaluation

To evaluate the generalization ability and tactical performance of the models, we construct a standardized benchmark opponent pool. This evaluation pool contains

eight opponent policies covering diverse engagement styles, including aggressive, evasive, and random-maneuvering behaviors. These benchmark opponents are selected snapshots obtained from various training iterations and configurations. Their diverse tactical styles are induced by adjusting the relative weights of reward terms and varying the initial engagement geometries during their training. For instance, policies trained from advantageous initial positions tend to develop aggressive offensive tactics, whereas those initialized in disadvantageous positions learn more conservative or evasive behaviors. To provide a more intuitive characterization of the benchmark opponent pool, representative engagement scenarios of the eight opponents are illustrated in Figure 7, where the red aircraft denotes the benchmark opponent. As shown in the figure, the selected opponents exhibit different tactical preferences, including spiral offensive maneuvers, weaving attacks, and evasive disengagement behaviors.

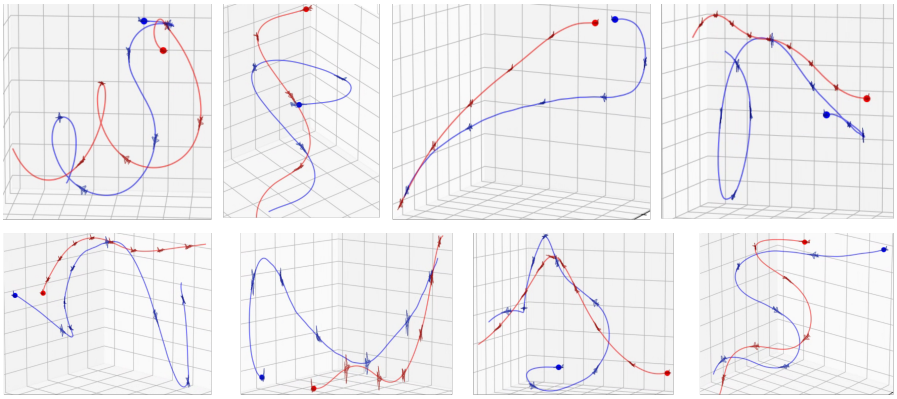


Fig. 7 Representative engagement scenarios of the eight benchmark opponents (the red aircraft).

Five models are compared: the baseline model (Baseline), a model with curriculum learning only (BC), a model with the dynamic opponent pool only (BD), and two versions of the BDC method, namely the multilayer perceptron version (BDC-MLP) and the long short-term memory version (BDC-LSTM). During evaluation, each model is matched against all eight opponents, with 1000 independent Monte Carlo simulations conducted for each matchup. In these simulations, the Monte Carlo sampling is specifically applied to the initial states of the aircraft, ensuring that both the starting position and attitude are randomly initialized for every episode to thoroughly assess the model's robustness.

Figure 8 shows the win/draw/loss distributions on the benchmark opponent pool. The Baseline is almost unable to secure victories, achieving a win rate of only 0.42%, with the vast majority of engagements ending in draws. This indicates that training only against fixed simple opponents may lead to policy overfitting. After introducing curriculum learning and the dynamic opponent pool separately, the win rates increase to 22.4% and 33.3%, respectively, showing that both components can independently improve generalization performance. The BDC method further improves the overall adversarial performance, with BDC-MLP and BDC-LSTM achieving win rates of

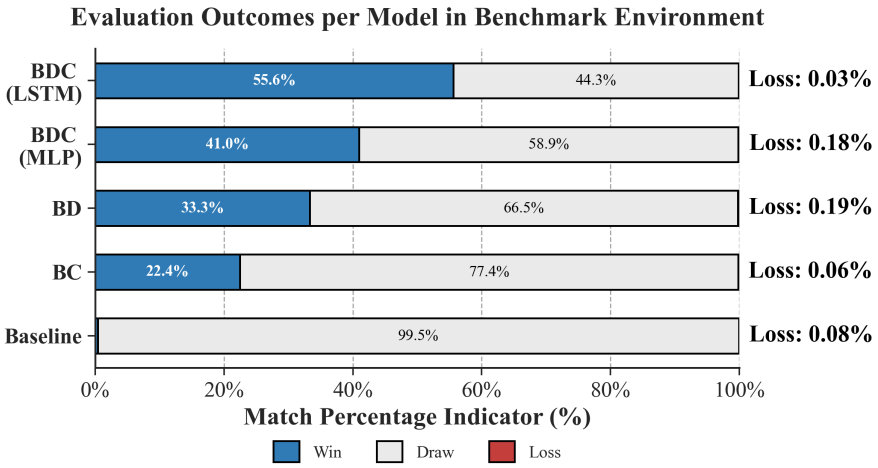


Fig. 8 Win, draw, and loss rates of different models against the benchmark opponent pool over 8×1000 simulated engagements per model.

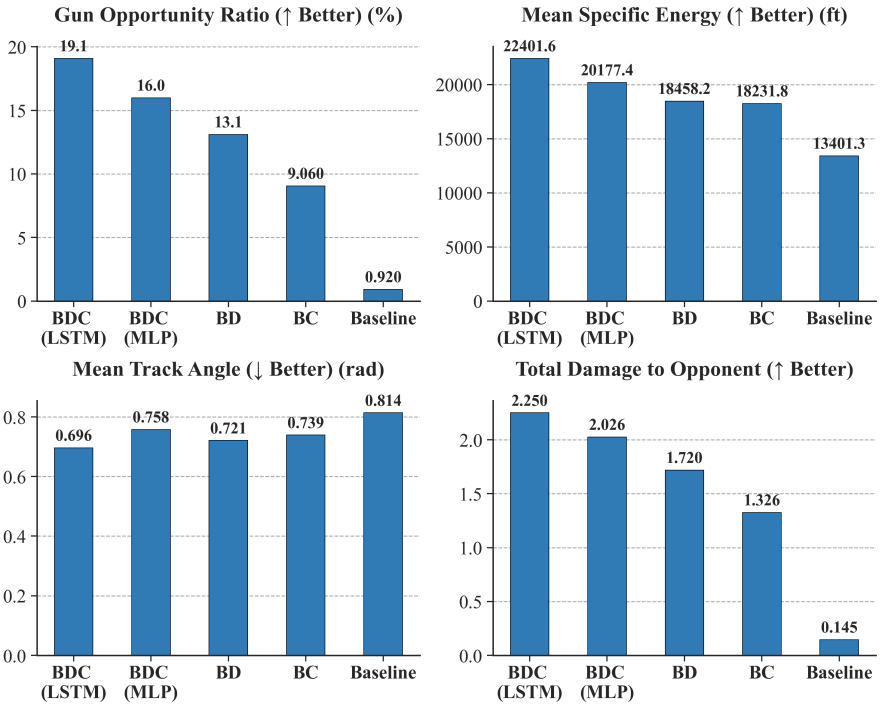


Fig. 9 Comparison of key air-combat metrics, including gun opportunity ratio, mean specific energy, mean track angle, and total damage to the opponent.

41.0% and 55.6%, respectively. The BDC-LSTM model therefore outperforms the baseline by approximately 55.2 percentage points.

Fig. 9 further compares four key air-combat metrics. The gun opportunity ratio is defined as the proportion of the total engagement time during which the opponent remains within the aircraft's weapons engagement zone. The mean specific energy denotes the aircraft's average specific energy (calculated as $E_s = h + \frac{V_T^2}{2g}$, where h is the altitude, V_T is the true velocity, and g is the gravitational acceleration) over the course of the engagement. Maintaining a higher level of specific energy is important in dogfight scenarios [31, 32], as it reflects the policy's ability to manage the conversion between kinetic and potential energy, which is often critical for gaining positional and tactical advantages during engagement. Mean track angle refers to the average track angle of the aircraft, where a smaller value indicates stronger target-following capability. The total damage to opponent denotes the average total damage to opponent in the whole engagement.

First, in terms of gun opportunity ratio, BDC-MLP and BDC-LSTM reach 16.0% and 19.1%, respectively, which are higher than 0.92% achieved by the baseline. This indicates that the BDC method can maintain the opponent within an effective firing envelope more consistently. Second, for mean specific energy, the BDC method outperforms the compared methods, with BDC-LSTM reaching 22401 ft, suggesting better energy retention during high-maneuver engagements. Third, in mean track angle, BDC-LSTM obtains the best result (0.696 rad), indicating that it can maintain a more favorable target-pointing condition in complex engagements. Lastly, the BDC method also achieves the highest score in the total damage to opponent metric, inflicting the most damage on the adversary.

4.3.2 Cross-Play Evaluation

In addition to the benchmark opponent pool evaluation, we further conduct cross-play experiments to compare the relative combat performance of different models. The models are evaluated in a round-robin manner, with each model competing against every other model. We evaluated the win rate of each model and the difference in remaining HP between the ego and the opponent at the end of each engagement. The results are shown in Figure 10.

In Figure 10, the color indicates the HP difference between the row model and the column model. The results show that models using BDC method achieve higher win rates and positive ΔHP values against all ablation models, whereas the baseline is consistently at a disadvantage when facing the other methods. Introducing curriculum learning alone or the dynamic opponent pool alone can both improve policy quality, but neither matches the BDC method. Furthermore, a comparison across network architectures reveals that LSTM-based policies outperform their MLP counterparts.

We can also present the detailed engagement results between BDC-LSTM and BDC-MLP. Since both are trained with the proposed method and therefore have similar performance, the proportion of draws in their direct encounters is high. To further analyze their differences, Figure 11 presents the distributions of remaining HP and specific energy in cross-play.

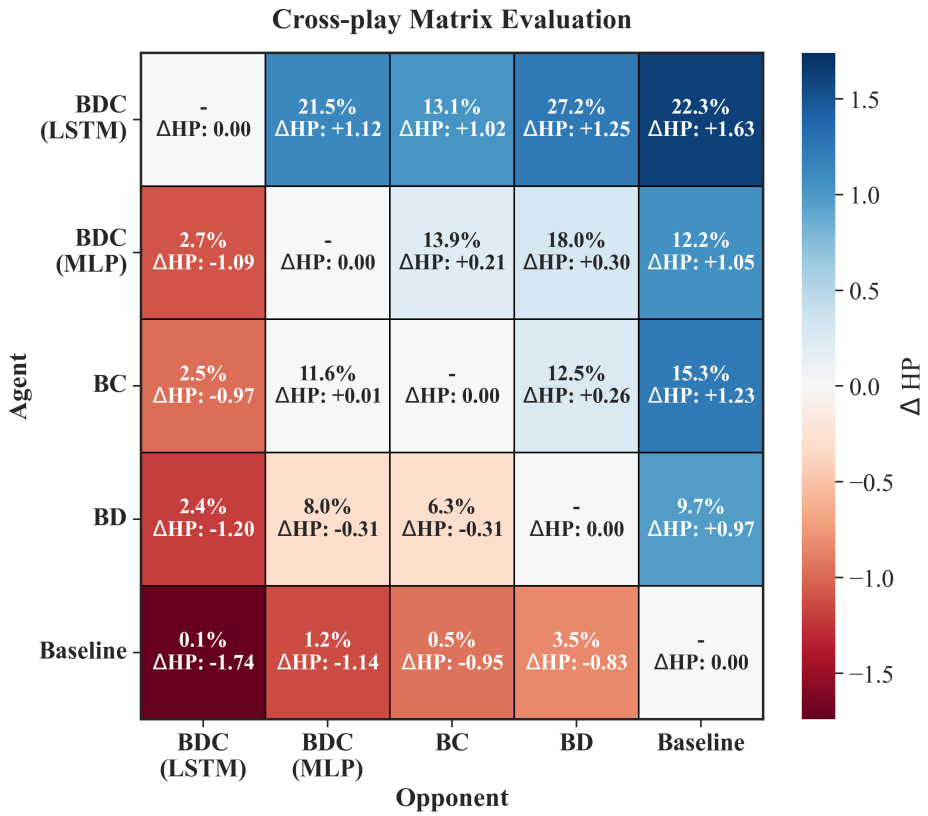


Fig. 10 Cross-play results among the ablation models. Each cell reports the win rate and the remaining-HP difference (ΔHP) of the row agent against the column agent.

As shown in Figure 11, BDC-LSTM outperforms BDC-MLP in direct competition. Their win rates are 6.2% and 0.9%, respectively, and BDC-LSTM also attains a higher average remaining HP (2.6 vs. 2.3). Meanwhile, the average specific energy of BDC-LSTM is approximately 20388 ft, which is higher than the 13499 ft of BDC-MLP, indicating better energy management during engagements.

In addition, the two models exhibit different energy distribution patterns. BDC-MLP shows a concentrated unimodal distribution, whereas BDC-LSTM presents a bimodal characteristic. This suggests that the LSTM architecture may help the policy develop more differentiated energy-utilization modes under different engagement situations. These findings suggest that incorporating temporal memory may improve decision-making flexibility in adversarial combat.

4.4 Engagement Demonstration

To better analyze the agent's maneuvering decisions, we developed a three-dimensional trajectory visualization with six-degree-of-freedom state information (3D attitude

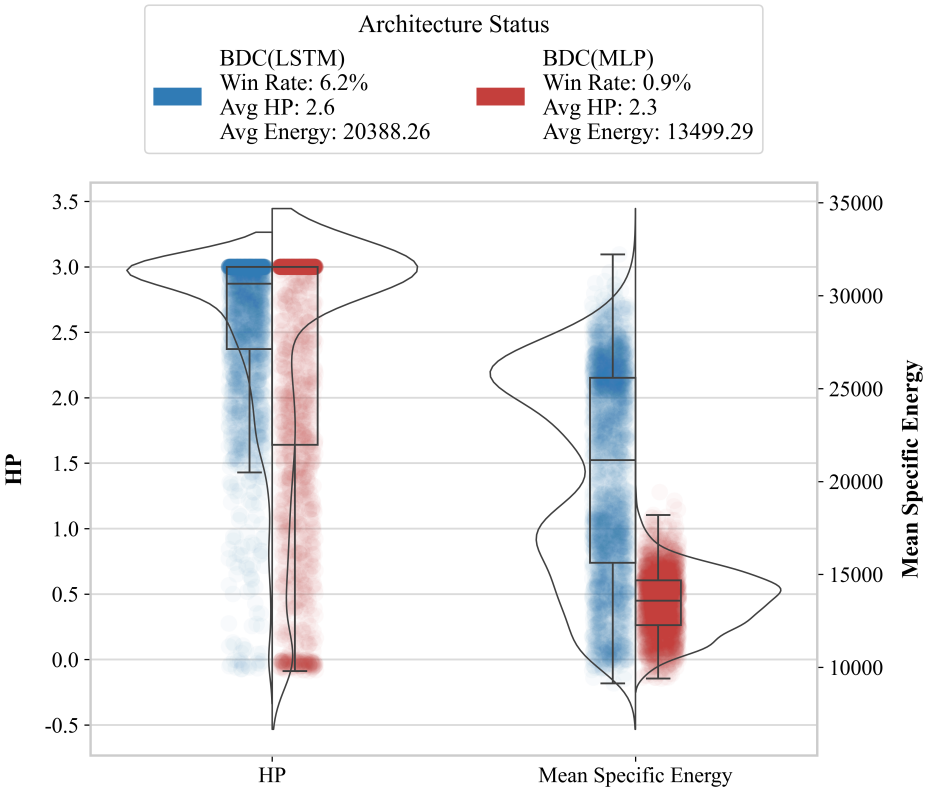


Fig. 11 Distributions of remaining health points (left) and specific energy (right) in direct cross-play between BDC-LSTM and BDC-MLP. Violin width indicates data density. The embedded boxplot shows the median and interquartile range (IQR), with whiskers extending to 1.5 × IQR. Overlaid scatter points represent individual samples. Together, these elements visualize the distribution shape, dispersion, and sample-level variability of both metrics.

plot). This section presents four representative engagement patterns observed in direct matches between BDC-LSTM (blue) and BDC-MLP (red).

Figure 12 illustrates four representative basic fighter maneuvers:

1. **Rolling Scissors**: As shown in Figure 12(a), after a close-range merge, both aircraft attempt to gain a favorable firing position behind the opponent, and the engagement develops into a typical rolling scissors maneuver.
2. **Defensive Barrel Roll**: As shown in Figure 12(b), the blue aircraft performs a defensive barrel roll while being pursued. By following a three-dimensional helical trajectory, it reduces the closure rate and changes the relative geometry, thereby inducing an overshoot by the red aircraft and improving its own tactical situation.
3. **Low Yo-Yo**: As shown in Figure 12(c), when the pursuit distance begins to increase, the blue aircraft executes a dive to gain speed, shortens its path on a lower turning plane, and compresses the relative distance before pulling up again to recover an attacking posture.

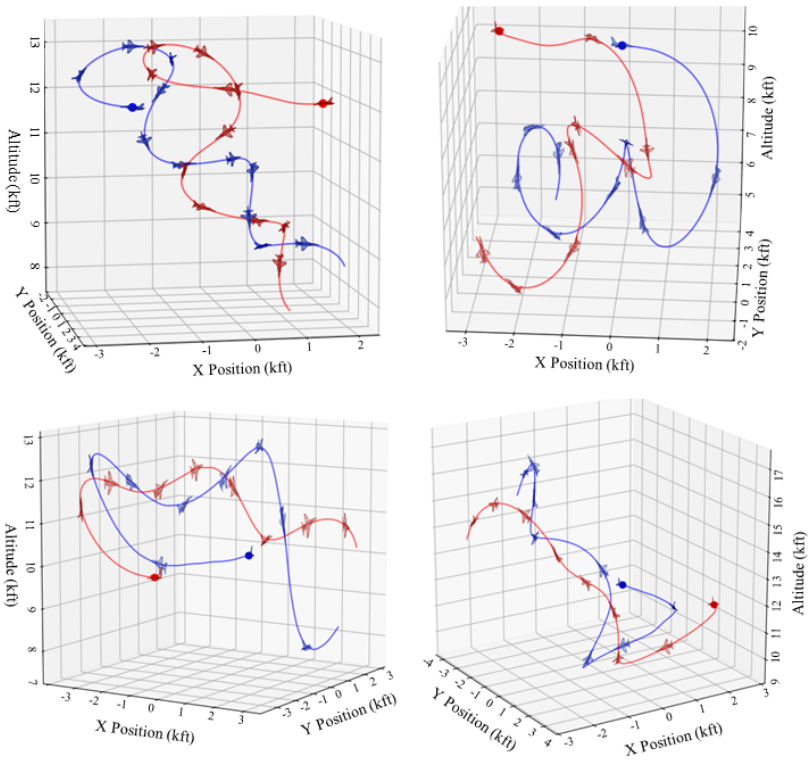


Fig. 12 Three-dimensional trajectory visualizations with 6-DOF aircraft attitudes during direct engagements between BDC-LSTM (blue) and BDC-MLP (red). Solid dots indicate the starting positions. The four panels show representative basic fighter maneuvers observed during the engagements: (a) rolling scissors, (b) defensive barrel roll, (c) low Yo-Yo, and (d) high Yo-Yo.

4. **High Yo-Yo**: As shown in Figure 12(d), when the blue aircraft enters the pursuit with a high closure rate, it does not simply follow the red aircraft in a planar turn. Instead, it deliberately climbs, converting part of its kinetic energy into potential energy, reducing the risk of overshoot and improving the turning geometry before re-entering a favorable firing position.

These representative cases suggest that the BDC method is able to spontaneously develop diverse maneuvering strategies according to relative position, speed, and energy state.

5 Conclusion

This paper proposes a reinforcement learning framework for within-visual-range autonomous air combat, providing a solution with generalization and robustness. Through the integration of curriculum learning and dynamic opponent pool, the proposed framework stabilizes training and mitigates cyclic dominance. Experimental

evaluations indicate that the proposed method yields competitive performance against baseline methods in terms of win rate and energy management. Ultimately, this work offers a viable baseline for addressing flight control and tactical decision-making in autonomous dogfighting.

Appendix A

The appendix provides supplementary implementation details for the proposed training framework. Table A1 summarizes the complete state observation space together with their corresponding physical lower and upper bounds. Table A2 lists the basic reward terms used for training the level-flight opponents, where $\sigma(\cdot)$ denotes the logistic function; u^*_{ail}, u^*_{thr}, and ϕ^* denote the target values of aileron input, throttle input, and roll angle, respectively; h_{low} and h_{high} denote the lower and upper boundaries of the allowable flight envelope; $k_a, k_t, k_r, k_l, k_h, k_s, k_{\Delta h}$ are the weighting coefficients of the corresponding reward terms; and s_l, s_h control the slope of the boundary penalty transition. Table A3 reports the main algorithm and network hyperparameters used for both the LSTM-based and MLP-based policies. Table A4 presents the environment and stage configuration.

References

[1] Strand, A., Gorton, P.R., Brathen, K.: Modeling air combat behavior for simulation-based pilot training: a survey of machine learning approaches. IEEE Access **13**, 113666–113685 (2025) https://doi.org/10.1109/ACCESS.2025.3580279

[2] Pope, A.P., Ide, J.S., Mićović, D., Diaz, H., Twedt, J.C., Alcedo, K., Walker, T.T., Rosenbluth, D., Ritholtz, L., Javorsek, D.: Hierarchical reinforcement learning for air combat at DARPA's AlphaDogfight trials. IEEE Transactions on Artificial Intelligence **4**(6), 1371–1385 (2023) https://doi.org/10.1109/TAI.2022.3222143

[3] Koch, W., Mancuso, R., West, R., Bestavros, A.: Reinforcement learning for UAV attitude control. ACM Trans. Cyber-Phys. Syst. **3**(2) (2019) https://doi.org/10.1145/3301273

[4] Bøhn, E., Coates, E.M., Reinhardt, D., Johansen, T.A.: Data-efficient deep reinforcement learning for attitude control of fixed-wing UAVs: field experiments. IEEE Transactions on Neural Networks and Learning Systems **35**(3), 3168–3180 (2024) https://doi.org/10.1109/TNNLS.2023.3263430

[5] O'Connell, M., Shi, G., Shi, X., Azizzadenesheli, K., Anandkumar, A., Yue, Y., Chung, S.-J.: Neural-Fly enables rapid learning for agile flight in strong winds. Science Robotics **7**(66), 6597 (2022) https://doi.org/10.1126/scirobotics.abm6597

Table A1 Complete state observation space variables and extremum boundaries.

State category	Variable name	Lower bound	Upper bound
Aircraft base states	h	-1400 ft	85000 ft
	θ	$-\pi/2$ rad	$\pi/2$ rad
	ϕ	$-\pi$ rad	π rad
	u	-2200 ft/s	2200 ft/s
	v	-2200 ft/s	2200 ft/s
	w	-2200 ft/s	2200 ft/s
	p	-2π rad/s	2π rad/s
	q	-2π rad/s	2π rad/s
	r	-2π rad/s	2π rad/s
Aircraft tracking states	heading	0 deg	360 deg
	a_x, a_y, a_z	–	–
	$a_{roll}, a_{pitch}, a_{yaw}$	–	–
	α	-180 deg	180 deg
	β	-180 deg	180 deg
	aileron, elevator, rudder	-1	1
	throttle Aug, engine thrust	–	–
	vtrue	0 ft/s	2200 ft/s
Relative geometry	D	0 ft	500000 ft
	θ_{tra}	$-\pi$ rad	π rad
	θ_{bea}, accounting roll/pitch	$-\pi$ rad	π rad
	θ_{ele}, accounting roll/pitch	$-\pi$ rad	π rad
	θ_{bea}, point mass	$-\pi$ rad	π rad
	θ_{ele}, point mass	$-\pi$ rad	π rad
	θ_{adv}	$-\pi$ rad	π rad
	V_c	-100000 ft/s	100000 ft/s
Opponent states	x_{oppo}, y_{oppo}	-100000 ft	100000 ft
	h_{oppo}	-1400 ft	85000 ft
	$\phi_{oppo}, \theta_{oppo}$	$-\pi, -\pi/2$ rad	$\pi, \pi/2$ rad
	heading_{oppo}	0 deg	360 deg
	$u_{oppo}, v_{oppo}, w_{oppo}$	-2200 ft/s	2200 ft/s
	$p_{oppo}, q_{oppo}, r_{oppo}$	-2π rad/s	2π rad/s
	$\alpha_{oppo}, \beta_{oppo}$	-180 deg	180 deg
	$\theta_{tra,oppo}$	$-\pi$ rad	π rad
	vtrue_{oppo}	0 ft/s	2200 ft/s
	$\theta_{bea,oppo}, \theta_{ele,oppo}$	$-\pi$ rad	π rad
Historical actions	$\delta_a, \delta_e, \delta_r, \delta_T$	-1	1

[6] Clarke, S.G., Hwang, I.: Deep reinforcement learning control for aerobatic maneuvering of agile fixed-wing aircraft. In: AIAA Scitech 2020 Forum. AIAA, Orlando, FL, USA (2020). https://doi.org/10.2514/6.2020-0136

[7] Clarke, R., Fletcher, L., East, S., Richardson, T.: Reinforcement learning derived high-alpha aerobatic manoeuvres for fixed wing operation in confined spaces.

Table A2 Basic reward terms used for training the level-flight opponents.

Reward term	Mathematical form
Aileron magnitude penalty	$r_{\text{ail}} = -k_a \lvert u_{\text{ail}} - u^*_{\text{ail}} \rvert$
Throttle penalty	$r_{\text{thr}} = -k_t \lvert u_{\text{thr}} - u^*_{\text{thr}} \rvert$
Roll attitude penalty	$r_{\text{roll}} = -k_r \lvert \phi - \phi^* \rvert$
Lower altitude penalty	$r_{\text{low}} = -k_l \left(1 - \sigma\left(\frac{h - h_{\text{low}}}{s_l}\right)\right)$
Upper altitude penalty	$r_{\text{high}} = -k_h \left(1 - \sigma\left(\frac{h_{\text{high}} - h}{s_h}\right)\right)$
Action smoothness penalty	$r_{\text{smooth}} = -k_s \sum_{i \in \{\text{ail,ele,rud}\}} \lvert u_i^{(t)} - u_i^{(t-1)} \rvert$
Altitude smoothness penalty	$r_{\Delta h} = -k_{\Delta h} \lvert h_t - h_{t-1} \rvert$

Table A3 Algorithm and network hyperparameters.

Hyperparameter	LSTM	MLP
Learning rate	5.0×10^{-5}	1.0×10^{-4}
Rollout steps	2048	2048
Batch size	128	128
Epochs	10	10
Gamma (γ)	0.99	0.99
GAE lambda (λ)	0.95	0.95
Clip range	0.2	0.2
Value function coefficient	0.5	0.5
Max grad norm	0.5	0.5
Share features extractor	False	True
LSTM hidden size	256	–
Network arch (policy)	[512, 512]	[12288]
Network arch (value)	[512, 512]	Shared

Algorithms **16**(8), 384 (2023) https://doi.org/10.3390/a16080384

[8] Chai, J., Chen, W., Zhu, Y., Yao, Z.-X., Zhao, D.: A hierarchical deep reinforcement learning framework for 6-DOF UCAV air-to-air combat. IEEE Transactions on Systems, Man, and Cybernetics: Systems **53**(9), 5417–5429 (2023) https://doi.org/10.1109/TSMC.2023.3270444

[9] Wang, Y., Li, H., Shen, Q.: A hierarchical multi-task and multi-agent assignment approach: learning DQN strategy from execution. IEEE Transactions on Automation Science and Engineering **22**, 14712–14722 (2025) https://doi.org/10.1109/TASE.2025.3560121

[10] Feng, X., Ma, Y., Zhao, L., Yang, H.: Autonomous decision making with reinforcement learning in multi-UAV air combat. In: 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 2874–2879. IEEE, Honolulu, HI,

Table A4 Environment and stage configuration.

Parameter	Value
Aircraft model	F-16
Observation clipping threshold	10.0
Random seed	42
Number of CPUs	30
Evaluation frequency	50,000 episodes
Battle step	10,000,000
Battle cycles	30
Number of evaluation episodes	50
Threshold timesteps	20,000,000
Update frequency	1,000 episodes

USA (2023). https://doi.org/10.1109/SMC53992.2023.10394123

[11] Selmonaj, A., Szehr, O., Del Rio, G., Antonucci, A., Schneider, A., Rüegsegger, M.: Hierarchical multi-agent reinforcement learning for air combat maneuvering. In: 2023 International Conference on Machine Learning and Applications (ICMLA), pp. 1031–1038 (2023). https://doi.org/10.1109/ICMLA58977.2023.00153

[12] Chen, Y.-H., Chiang, M.-H., Peng, C.-C.: Physics-informed reinforcement learning for air-to-air F16 target aiming. Advanced Engineering Informatics **69**, 103988 (2026) https://doi.org/10.1016/j.aei.2025.103988

[13] Berndt, J.S., Coconnier, B., De Marco, A., McLeod, S.: JSBSim - an Open Source Flight Dynamics Software Library. https://doi.org/10.5281/zenodo.20258621 . https://github.com/JSBSim-Team/jsbsim

[14] Li, B.-W., Tan, T., Li, J., Zhang, J.-W., Zhang, X.-R.: A progressive deep reinforcement learning approach for 6-DOF air combat decision-making. Journal of Computer Applications (2025) https://doi.org/10.19678/j.issn.1000-3428.0252480 . Online published

[15] Giral, F., Gomez, I., Le Clainche, S.: Intercepting unauthorized aerial robots in controlled airspace using reinforcement learning. Aerospace Science and Technology **168**, 110985 (2026) https://doi.org/10.1016/j.ast.2025.110985

[16] Yu, L., Li, Z., Ansari, N., Sun, X.: Hybrid transformer based multi-agent reinforcement learning for multiple unpiloted aerial vehicle coordination in air corridors. IEEE Transactions on Mobile Computing **24**(6), 5482–5495 (2025) https://doi.org/10.1109/TMC.2025.3532204

[17] Yang, Q., Li, D., Huang, J., Zhou, P., Zhang, S.: Close-range air combat autonomous maneuver decision-making of UAV based on TD3-Transformer. International Journal of Aeronautical and Space Sciences (2025) https://doi.org/

10.1007/s42405-025-01079-2

[18] Hsu, C.D., Jeong, H., Pappas, G.J., Chaudhari, P.: Scalable reinforcement learning policies for multi-agent control. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4785–4791. IEEE, Prague, Czech Republic (2021). https://doi.org/10.1109/IROS51168.2021.9636344

[19] Bae, J.H., Jung, H., Kim, S., Kim, S., Kim, Y.-D.: Deep reinforcement learning-based air-to-air combat maneuver generation in a realistic environment. IEEE Access **11**, 26427–26440 (2023) https://doi.org/10.1109/ACCESS.2023.3257849

[20] Wang, L., Zheng, S., Tai, S., Liu, H., Yue, T.: UAV air combat autonomous trajectory planning method based on robust adversarial reinforcement learning. Aerospace Science and Technology **153**, 109402 (2024) https://doi.org/10.1016/j.ast.2024.109402

[21] Li, S., Zuo, R., Liu, B., Liu, P., Zhao, Y.: An imitative reinforcement learning framework for autonomous dogfight. Preprint at https://arxiv.org/abs/2406.11562 (2025)

[22] Kong, W.-r., Zhou, D.-y., Du, Y.-j., Zhou, Y., Zhao, Y.-y.: Hierarchical multi-agent reinforcement learning for multi-aircraft close-range air combat. IET Control Theory & Applications **17**(13), 1840–1862 (2023) https://doi.org/10.1049/cth2.12413

[23] Han, Y., Piao, H., Hou, Y., Sun, Y., Sun, Z., Zhou, D., Yang, S., Peng, X., Fan, S.: Deep relationship graph reinforcement learning for multi-aircraft air combat. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE, Padua, Italy (2022). https://doi.org/10.1109/IJCNN55064.2022.9892208

[24] Selmonaj, A., Szehr, O., Del Rio, G., Antonucci, A., Schneider, A., Rüegsegger, M.: Hierarchical multi-agent reinforcement learning for air combat maneuvering. In: 2023 International Conference on Machine Learning and Applications (ICMLA), pp. 1031–1038. IEEE, Jacksonville, FL, USA (2023). https://doi.org/10.1109/ICMLA58977.2023.00153

[25] Han, H., Cheng, J., Lv, M., Duan, H.: Augmenting the robustness of tactical maneuver decision-making in unmanned aerial combat vehicles during dogfights via prioritized population play with diversified partners. IEEE Transactions on Aerospace and Electronic Systems **61**(5), 12892–12907 (2025) https://doi.org/10.1109/TAES.2025.3578294

[26] Chen, C., Song, T., Mo, L., Lv, M., Lin, D.: Autonomous dogfight decision-making for air combat based on reinforcement learning with automatic opponent sampling. Aerospace **12**(3), 265 (2025) https://doi.org/10.3390/aerospace12030265

[27] Gorren: Gym-jsbsim, v1.0.0, GitHub. https://github.com/Gor-Ren/gym-jsbsim

[28] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. Preprint at https://arxiv.org/abs/1707.06347 (2017)

[29] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-Baselines3: reliable reinforcement learning implementations. Journal of Machine Learning Research **22**(268), 1–8 (2021)

[30] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997) https://doi.org/10.1162/neco.1997.9.8.1735

[31] Neufeld, J., Watson, George M. Jr, Chenoweth, D. (eds.): Technology and the Air Force: a Retrospective Assessment. Air Force History and Museums Program, United States Air Force, Washington, DC (1997)

[32] Boyd, J.R., Christie, T.P., Gibson, J.E.: Energy-maneuverability. Air Proving Ground Center Report APGC-TR-66-4 (1966)